DSA PROJECT DESCRIPTION - MINESWEEPER - A22

HOW TO START:

Run the program (README.md in the project folder explains how to build and run it).
The controls and the rules below are shown first.
Choose a level: type 1 for Easy (7 * 7), 2 for Medium (9 * 9) or 3 for Hard (11 * 11) and press Enter.
The board appears with every tile covered by #.
When a game ends, type 1 to play again or 2 to exit.

CONTROLS:

"The game is played purely using the keyboard as a controller."

SELECT: Enter Row and Col to select a tile. Type the row number and press Enter, then the column number and press Enter. Rows and columns are counted from 0, as numbered along the top and the left side of the board.
DIG: Select and enter 'd' to dig the selected tile.
FLAG: Select and enter 'f' to flag the selected tile.
UNFLAG: Select a flagged tile and flag it again to unflag it.
Type 'd' and 'f' as small letters and press Enter after them.

ON THE BOARD:

is a covered tile.
1 to 8 is a clue on a dug tile.
A blank dug tile has no mine around it.
F is a flag (in the Windows console the flag is an arrow symbol).
* is a mine. Mines are shown only when you lose.
"Flags Left" above the board is the number of mines minus the flags you have placed.

HOW TO PLAY:

Start with digging any tile. Your first dig is always safe, because the mines are placed only after it and never on that tile.

When you DIG you get a CLUE as a number in the tile.
The number shows how many mines are hidden under the adjacent tiles, that is the 8 tiles touching it (up, down, left, right and the 4 diagonals).
If there is no number in a dug tile, all tiles around it are clear, so the game digs them for you. If one of them is also empty, the tiles around it are dug too, so a whole clear area can open at once. A flag on a tile dug this way is removed and given back.
Mark any suspected mine tile as FLAG. A flag is only a reminder for you: it does not stop you from digging that tile.
If you think your suspected mine tile is clear, UNFLAG it.

You cannot dig or flag any tile which is already dug.

If you dig all the clear tiles without digging any mine tiles, you win. You do not have to flag the mines to win.
But be careful not to dig any mine tile,
it will trigger all the mines, the board will show every mine and you will lose the game.
Best Of Luck!

There are three levels for this game (the game menu calls them Easy, Medium and Hard):
Beginner (Easy) - 7 * 7 Board and 7 Mines
Intermediate (Medium) - 9 * 9 Board and 13 Mines
Advanced (Hard) - 11 * 11 Board and 22 Mines

Probability of finding a mine under a random tile (mines / tiles):
Beginner level - 7/49 (0.14)
Intermediate level - 13/81 (0.16)
Advanced level - 22/121 (0.18)

Following are the 11 main functions used in the code, in code order:

void gameload(int limit):

To load the linked board structure according to difficulty of game: one node for every tile, each linked to its 8 neighbouring tiles.

void display(int limit):

Displays the game layout which includes flag count and game board. It clears the screen and draws the board again after every move.

int subcountbomb(struct ground *g):

Just a sub function for countbomb() to reduce code redundancy. It checks one neighbouring tile and returns 1 if it has a mine, otherwise 0.

char countbomb(struct ground *g):

Counts the mines around the tile to be dug and returns that number as a character '1' to '8', or a blank when there are none.
This number is displayed in game board as a hint for players.

void digaround(struct ground *);

Declared here first so that subdigaround() can call digaround(), which is written after it.

void subdigaround(struct ground *g):

Just a sub function for digaround() to reduce code redundancy.
Digs one neighbouring tile if it is still covered and has no mine: calls countbomb() for it, gives back its flag if it had one, and if it has no mine around it, calls digaround(g) for it.

void digaround(struct ground *g):

Digs all 8 tiles around a tile which has no mine around it, that is g[row][col]->inground == ' '.
Calls subdigaround() for each of the 8 neighbours, which calls countbomb() for that neighbour.
If that neighbour's inground is also ' '
subdigaround() calls digaround() for it.
So, it is a recursive function.
It prevents game from getting boring as digging tiles without mines around one by one makes game boring and tedious to play.

void randombomb(int limit,int totbomb,int row,int col):

Sets mines at random tiles by selecting a random row and col for each mine with help of rand() and srand() functions. In the original code this function was named random(); it was renamed because random() is also the name of a standard C library function on Linux and macOS.

It makes sure that no two mines are on the same tile and that no mine is on the first dug tile (row, col).
By this function game gets a new board on every execution and every time when played again.

void validate(int *data):
It reads a whole number. If anything else is typed (for example a letter), it shows "Invalid Input" and asks again.

void playgame(int limit):
Decides totbomb (number of mines) on basis of limit: 7, 13 or 22.
Controls while loop which ends (1) when all tiles except mines are dug or (2) any mine is dug.
In case 1 you win and in case 2 you lose.
Asks row and col for selected tile, and asks again if they are outside the board.
Also asks player to dig or flag and controls flag and dig count.
When dig is selected it
Calls randombomb(limit,totbomb,row,col) for the first dig only.
If the tile is already dug, asks for another tile.
If the tile has a mine, shows all mines and the game is lost.
Otherwise (giving back its flag if it had one):
Calls g[row][col]->inground = countbomb(g[row][col])
g[row][col]->groundptr = &g[row][col]->inground
If g[row][col]->inground == ' '
Calls digaround(g[row][col])
dig++
When flag is selected
If the tile is already dug, asks for another tile.
If not flagged
g[row][col]->onground=16
flag--
If already flagged then it is unflagged
g[row][col]->onground='#'
flag++

Calls display(limit) after every move.
At the end prints "You Win!!!" or "You Lose. Try Next Time!".

void emptymemory(int limit):

It frees the memory each time game ends.
So that memory is not wasted if players play game again and again without terminating program.
Resets dig and flag to 0.

int main(): The driver function

Shows the controls and rules, then gets limit (difficulty level): 7, 9 or 11.
Calls gameload(limit)
Calls display(limit)
Calls playgame(limit)
Calls emptymemory(limit)
Controls loop for play again and exit.

The code also has 6 small helper functions: set_color() and clear_screen() for colors and clearing the screen, clear_line() to skip the rest of a typed line, and print_border(), print_row_line() and tile_color() to draw the board.

This is the structure for the linked board (one node per tile):

```
struct ground{
    struct ground *up;
    struct ground *down;
    struct ground *left;
    struct ground *right;
    struct ground *upright;
    struct ground *upleft;
    struct ground *downright;
    struct ground *downleft;
    char inground;
    char onground;
    char *groundptr;
} *g[11][11];
```

The 8 pointers link a tile to its neighbours (NULL at the edge of the board).
inground is what is under the tile: '*' for a mine, '1' to '8' for a clue, or ' '.
onground is what covers the tile: '#', or the flag (character 16).
groundptr points to onground while the tile is covered and to inground once it is dug, so the board shows whichever one it points to.

g is an 11 * 11 array of pointers to tiles, big enough for the Hard board. Smaller boards use its top left part.

And these are global variables used in all functions:
int arrayr[22],arrayc[22],dig=0,flag=0;
arrayr and arrayc hold the row and column of each mine, dig counts the dug tiles and flag is the "Flags Left" count.

In the game console colors are used to make it more fun.
set_color(X)
Here X is number for color. On Windows set_color() calls SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), X). On other systems it prints the matching ANSI color code.
Used colors are given below with their respective numbers:

1 blue
2 green
4 red
5 purple
6 dark yellow
7 white
8 grey
11 cyan blue
12 light red
13 light purple

Each clue number 1 to 8 has its own color, mines and flags are red, and the row and column numbers and grid lines are grey.

~ Priyanshu Maniar